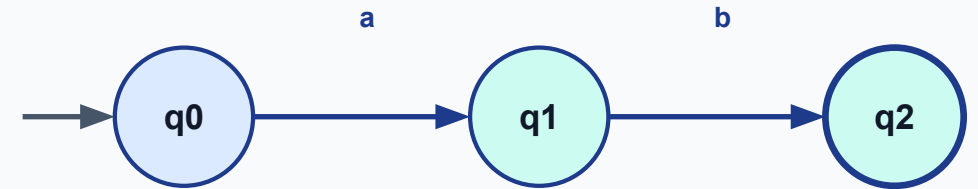


Linguagens Formais e Compiladores

Autômatos finitos determinísticos e não determinísticos com transições vazias

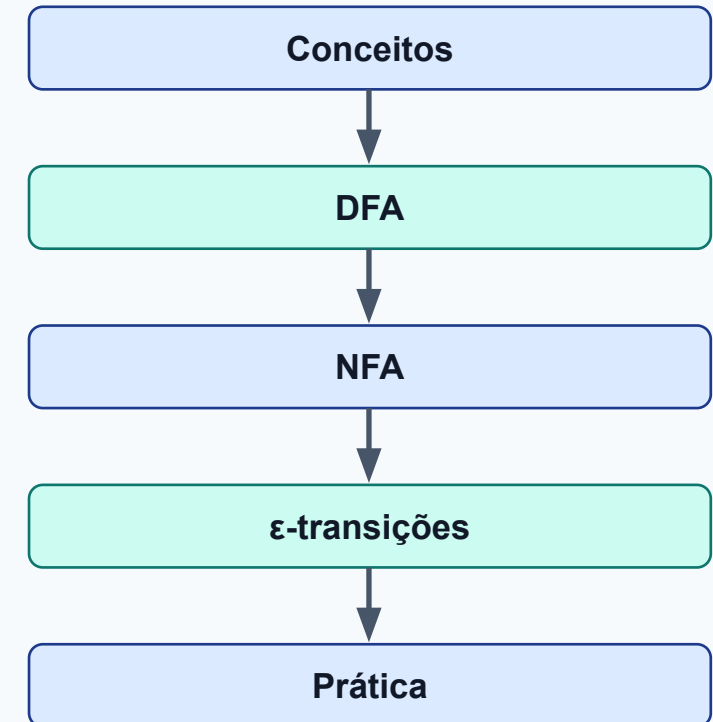


DFA, NFA e ϵ -transições

Objetivos da aula

Panorama do encontro

Ao final desta aula, o estudante deverá compreender o que é um autômato finito, distinguir DFA e NFA, interpretar transições vazias, simular entradas passo a passo e relacionar esses modelos à análise léxica de compiladores. A aula combina explicação conceitual, diagramas, tabelas e prática com ferramentas gratuitas.



Retomando a ideia de linguagem

Alfabeto, palavra e linguagem

Em linguagens formais, um alfabeto é um conjunto finito de símbolos, uma palavra é uma sequência formada com esses símbolos e uma linguagem é um conjunto de palavras. O autômato finito será usado para responder uma pergunta prática: uma palavra pertence ou não pertence à linguagem definida?

$$\Sigma = \{a,b\}$$

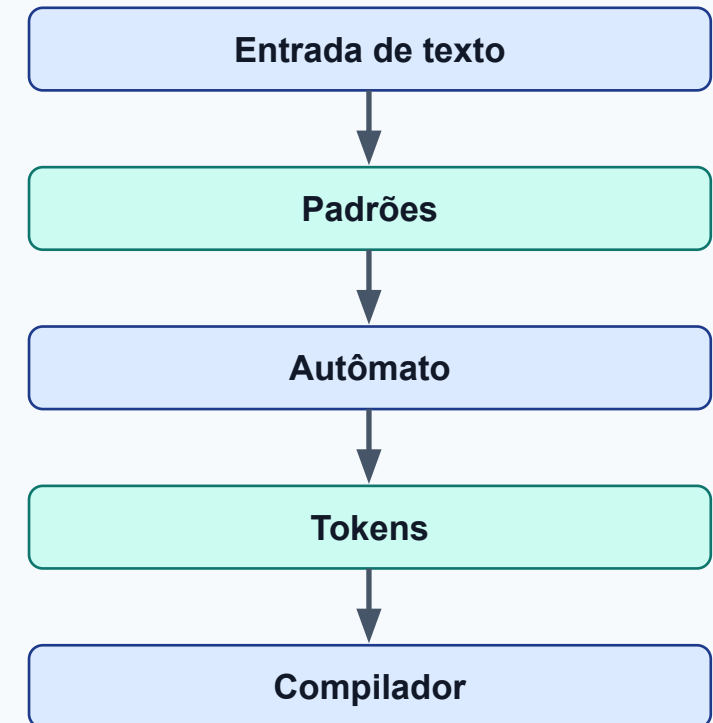
$$w = abba$$

$$L = \text{palavras aceitas}$$

Por que autômatos finitos?

Motivação em computação

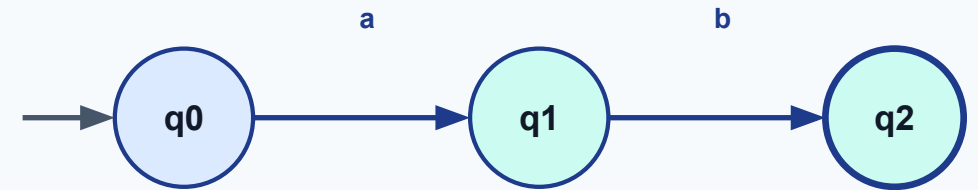
Autômatos finitos modelam sistemas que possuem memória limitada e mudam de situação conforme recebem símbolos de entrada. Eles aparecem em validadores, protocolos simples, busca de padrões, expressões regulares e analisadores léxicos. Em compiladores, ajudam a transformar caracteres do código-fonte em tokens organizados.



O que é um autômato finito?

Definição intuitiva

Um autômato finito é uma máquina abstrata composta por estados e transições. Ela lê uma palavra da esquerda para a direita, símbolo por símbolo. A cada leitura, muda de estado. Quando a entrada termina, a palavra será aceita se o autômato estiver em um estado final.



Máquina abstrata de estados

Componentes formais de um FA

Quíntupla $M = (Q, \Sigma, \delta, q_0, F)$

A definição formal de um autômato finito costuma usar cinco componentes: conjunto de estados Q , alfabeto Σ , função ou relação de transição δ , estado inicial q_0 e conjunto de estados finais F . Essa notação permite descrever o modelo com precisão, sem depender apenas do desenho.

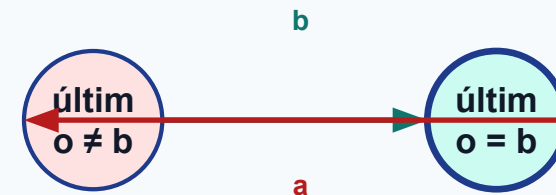
Componentes

Símbolo	Significado	Exemplo
Q	estados	$\{q_0, q_1\}$
Σ	alfabeto	$\{a, b\}$
δ	transição	$\delta(q_0, a) = q_1$
q_0	inicial	q_0
F	finais	$\{q_1\}$

Estados: memória resumida

O papel dos estados

Um estado representa uma situação parcial da leitura. Ele resume o que já foi observado sem guardar necessariamente toda a palavra. Por exemplo, para reconhecer palavras que terminam em b, basta saber se o último símbolo lido foi b ou não. Essa memória resumida torna o modelo finito.

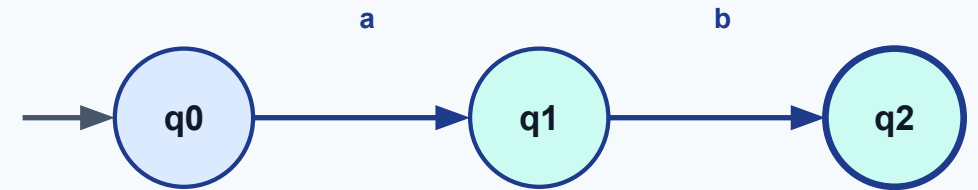


Memória finita

Transições: regras de mudança

Como as setas funcionam

Uma transição indica para onde o autômato deve ir ao ler determinado símbolo. Se ele está em q_0 e lê a , pode ir para q_1 . Se depois lê b , pode ir para q_2 . O desenho é uma forma visual de representar a função ou relação de transição.

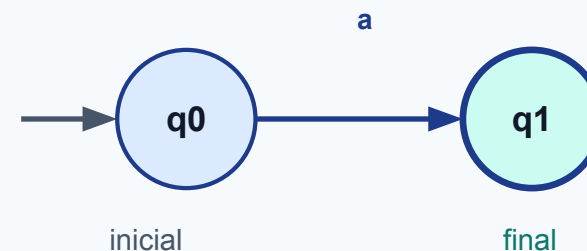


Transições rotuladas

Estado inicial e estados finais

Início e aceitação

O estado inicial indica onde a leitura começa. Ele costuma receber uma seta sem origem. Os estados finais, geralmente desenhados com círculo duplo, representam situações de aceitação. A entrada inteira precisa ser consumida; não basta passar por um estado final antes de terminar a palavra.



Leitura de uma palavra

Simulação passo a passo

Simular um autômato significa acompanhar o estado atual enquanto a palavra é lida. Começamos no estado inicial, consumimos o primeiro símbolo, aplicamos a transição correspondente e repetimos até o último símbolo. No final, verificamos se o estado alcançado pertence ao conjunto de estados finais.

Algoritmo de simulação

- 1 Começar em q_0
- 2 Ler o próximo símbolo
- 3 Aplicar a transição
- 4 Atualizar o estado
- 5 Verificar se é final

Aceitação e rejeição

Critério de decisão

Uma palavra é aceita quando existe uma execução válida que consome toda a entrada e termina em estado final. Ela é rejeitada quando a execução termina em estado não final ou quando não há transição possível para continuar. Em DFA, a execução é única; em NFA, pode haver várias possibilidades.

Exemplo ab^*

Entrada	Caminho	Resultado
a	$q_0 \rightarrow q_1$	aceita
ab	$q_0 \rightarrow q_1 \rightarrow q_1$	aceita
b	sem caminho	rejeita
ba	sem caminho	rejeita

DFA: autômato determinístico

Definição de DFA

No DFA, para cada par formado por estado e símbolo de entrada existe no máximo uma próxima transição definida. Isso significa que a leitura segue um único caminho, sem escolhas. Essa propriedade facilita implementação em código, depuração e uso em analisadores léxicos reais.

DFA: uma opção

δ	a	b
q0	q1	q0
q1	q1	q2
q2	q1	q0

Função de transição no DFA

$\delta: Q \times \Sigma \rightarrow Q$

A transição de um DFA pode ser vista como uma função matemática. Para cada estado e símbolo, a função retorna um único estado de destino. Quando a função está total, todos os pares possuem resposta. Quando está parcial, algumas entradas podem parar a execução por falta de transição.

$$\delta(q_0, a) = q_1$$

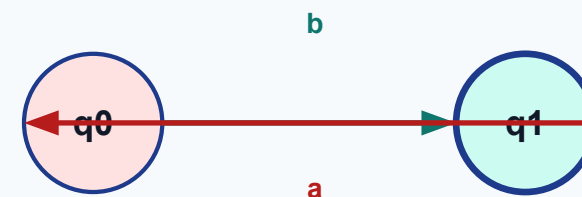
$$\delta(q_1, b) = q_2$$

um resultado

Exemplo de DFA: termina com b

Linguagem L = palavras sobre $\{a,b\}$ que terminam em b

Considere a linguagem formada por todas as palavras sobre $\{a,b\}$ cujo último símbolo é b . O autômato precisa apenas saber se o símbolo mais recente foi b . O estado q_1 será final porque representa a situação “a palavra lida até agora termina em b ”.



Simulando a entrada aab

Rastreamento de execução em DFA

Para simular a palavra aab, começamos em q0. Lemos a e permanecemos em q0; lemos outro a e continuamos em q0; lemos b e seguimos para q1. Como q1 é final e a entrada acabou, a palavra aab é aceita pela linguagem.

Rastreamento

Passo	Símbolo	Estado
0	—	q0
1	a	q0
2	a	q0
3	b	q1 ✓

Tabela de transição

Outra forma de representar o mesmo DFA

O mesmo autômato pode ser representado por diagrama ou por tabela. A tabela facilita implementação e testes, porque cada linha corresponde a um estado e cada coluna corresponde a um símbolo. Em algoritmos de compiladores, essa forma pode ser usada como estrutura de decisão.

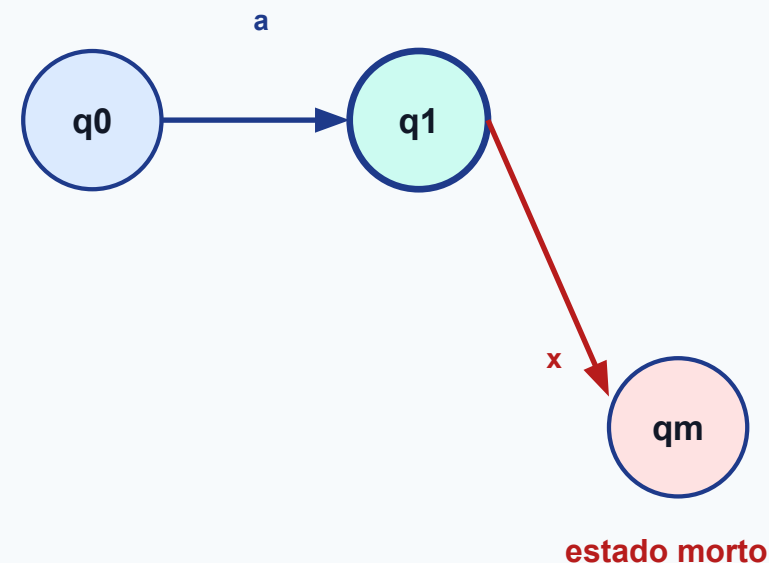
DFA termina com b

Estado	a	b
q0	q0	q1
q1	q0	q1

Estado morto e DFA total

Tratamento de entradas inválidas

Um estado morto representa uma situação da qual não há mais possibilidade de aceitação. Ele é útil quando queremos uma função de transição total, isto é, definida para todo estado e símbolo. Muitos livros usam esse recurso para evitar transições ausentes e tornar o DFA completamente especificado.



DFA e análise léxica

Aplicação em compiladores

Na análise léxica, padrões como identificadores, números e operadores podem ser reconhecidos por autômatos finitos. O analisador lê o código-fonte, agrupa caracteres em lexemas e produz tokens. Por isso, DFA é uma base importante para construir scanners eficientes em compiladores e interpretadores.

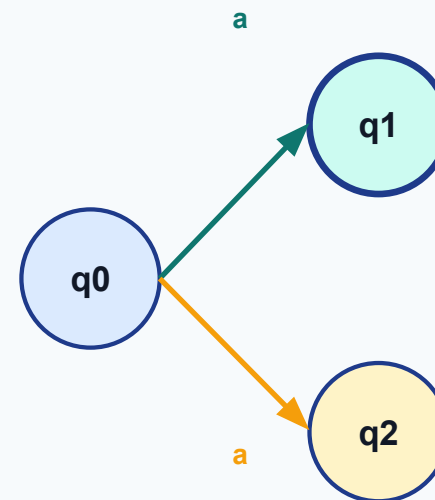
Apoio: Python re — <https://docs.python.org/3/library/re.html>



NFA: autômato não determinístico

Definição de NFA

No NFA, um mesmo estado pode ter mais de uma transição para o mesmo símbolo, ou pode possuir transições vazias. Durante a leitura, imaginamos que o autômato pode explorar caminhos alternativos. A palavra é aceita se pelo menos um desses caminhos consome a entrada e termina em estado final.



Relação de transição no NFA

$$\delta: Q \times \Sigma \rightarrow P(Q)$$

No NFA, a transição deixa de retornar apenas um estado e pode retornar um conjunto de estados possíveis. A notação $P(Q)$ indica o conjunto das partes de Q . Em termos práticos, a execução acompanha um conjunto de estados ativos depois de cada símbolo lido.

$$\delta(q_0, a) = \{q_1, q_2\}$$

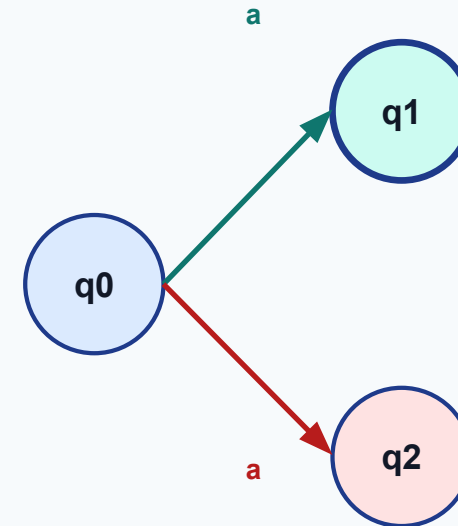
$$\delta(q_1, b) = \{q_1\}$$

conjunto de destinos

Como o NFA aceita uma palavra?

Aceitação por existência de caminho

A palavra é aceita por um NFA quando existe pelo menos uma sequência de escolhas que leva a um estado final após consumir toda a entrada. Não é necessário que todos os caminhos aceitem. Se um caminho falha, outro ainda pode aceitar, e isso torna o NFA conveniente para modelar alternativas.

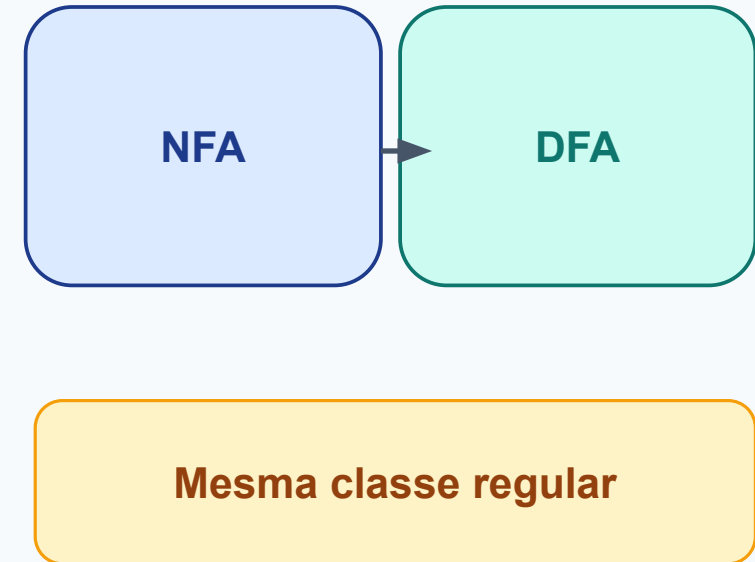


um caminho aceita ✓

NFA é mais poderoso que DFA?

Expressividade equivalente

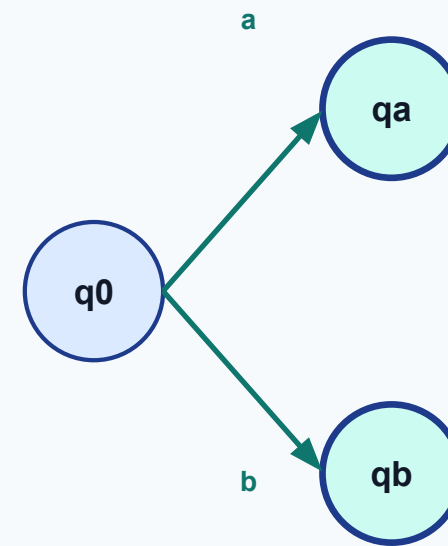
Embora pareça mais flexível, o NFA não reconhece linguagens além das linguagens regulares. Todo NFA possui um DFA equivalente, capaz de reconhecer exatamente o mesmo conjunto de palavras. A vantagem do NFA está na simplicidade de construção e na proximidade com expressões regulares.



Exemplo de NFA com escolha

Linguagem que contém a ou b no início

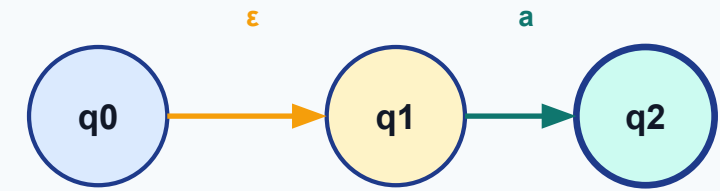
Considere um NFA que, a partir de q_0 , possui duas transições possíveis: uma para reconhecer palavras que começam com a e outra para palavras que começam com b. O modelo representa uma alternativa natural. Em vez de desenhar muitos casos separadamente, podemos deixar o NFA escolher um caminho.



Transições vazias

λ ou ϵ -transições

Uma transição vazia permite mudar de estado sem consumir símbolo da entrada. Em diferentes livros e ferramentas, ela pode aparecer como λ ou ϵ . Esse recurso é especialmente útil para combinar partes de autômatos, representar escolhas e construir autômatos a partir de expressões regulares.



muda sem ler entrada

ϵ não é símbolo consumido

Cuidado conceitual

A transição vazia não lê nenhum caractere. Se a entrada for a, o autômato pode sair de q_0 para q_1 por ϵ e ainda terá a palavra inteira para ler. Esse detalhe evita confundir a palavra vazia com um símbolo escrito na entrada. ϵ representa ausência de consumo.

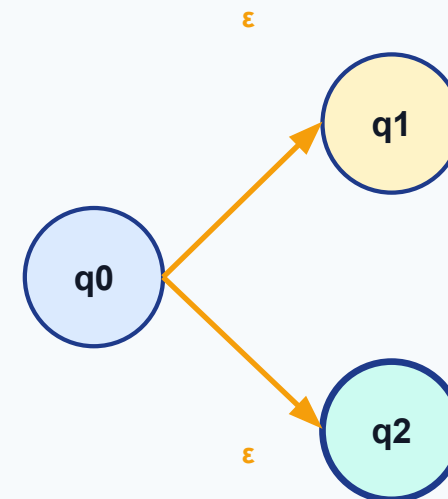
Rastreamento ϵ

Ação	Entrada restante	Estado
início	a	q_0
ϵ	a	q_1
lê a	—	q_2 ✓

Fecho- ϵ

Conjunto de estados alcançáveis sem consumir entrada

O fecho- ϵ de um estado é o conjunto de estados que podem ser alcançados usando apenas transições vazias, incluindo o próprio estado. Esse conceito é importante para simular NFAs com ϵ e para converter esse tipo de autômato em um DFA equivalente.

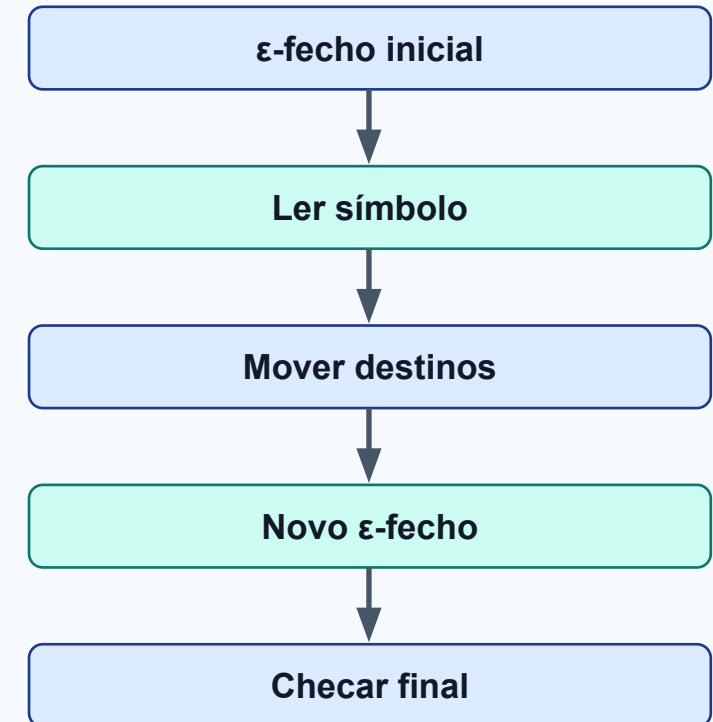


$$\epsilon\text{-fecho}(q_0) = \{q_0, q_1, q_2\}$$

Simulando NFA com ϵ

Estados ativos

Na simulação de um NFA com transições vazias, primeiro calculamos os estados alcançáveis por ϵ . Depois lemos um símbolo e avançamos para todos os destinos possíveis. Em seguida, calculamos novamente o fecho- ϵ . O processo continua até consumir a entrada e verificar se algum estado ativo é final.



DFA versus NFA

Comparação operacional

DFA possui um único caminho de execução e é direto para implementar. NFA pode ter várias escolhas e transições vazias, sendo mais fácil para modelar padrões compostos. Apesar dessa diferença operacional, ambos reconhecem exatamente linguagens regulares, e a conversão NFA para DFA preserva a linguagem.

Comparação

Critério	DFA	NFA
Caminho	único	vários
δ	estado	conjunto
ϵ	não	pode ter
Uso	execução	modelagem

Conversão de NFA para DFA

Construção dos subconjuntos

A conversão de NFA para DFA cria estados que representam conjuntos de estados do NFA. Cada estado do novo DFA indica onde o NFA poderia estar após ler certa parte da entrada. Assim, eliminamos as escolhas, porque o conjunto de possibilidades passa a ser tratado como um único estado determinístico.



estado = conjunto

Passos da construção dos subconjuntos

Algoritmo resumido

Comece pelo conjunto contendo o estado inicial do NFA e seu fecho- ϵ . Para cada símbolo do alfabeto, calcule os destinos possíveis e aplique novamente o fecho- ϵ . Cada conjunto novo vira estado do DFA. Repita até não aparecerem novos conjuntos. Marque como finais os conjuntos que contêm estado final do NFA.

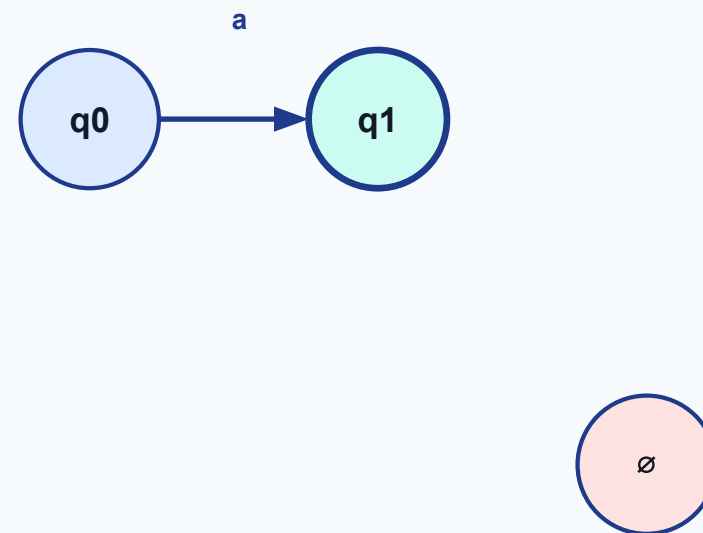
NFA $\rightarrow$ DFA

- 1 Calcular ϵ -fecho inicial
- 2 Mover por cada símbolo
- 3 Aplicar ϵ -fecho
- 4 Criar estado novo
- 5 Marcar finais

Exemplo de conversão

NFA simples para linguagem ab^*

Suponha um NFA em que q_0 chega a q_1 lendo a , e q_1 permanece em q_1 lendo b . Como q_1 é final, a linguagem aceita a , ab , abb e assim por diante. O DFA equivalente terá estados que representam os conjuntos alcançados após cada símbolo lido.



DFA inclui conjuntos e, se necessário, $\emptyset$

Quando surgem muitos estados?

Crescimento exponencial no pior caso

A conversão de NFA para DFA pode aumentar bastante o número de estados, porque cada estado do DFA pode representar um subconjunto de estados do NFA. No pior caso, um NFA com n estados pode gerar até 2^n estados no DFA, embora muitos exemplos didáticos gerem bem menos.

NFA: n estados

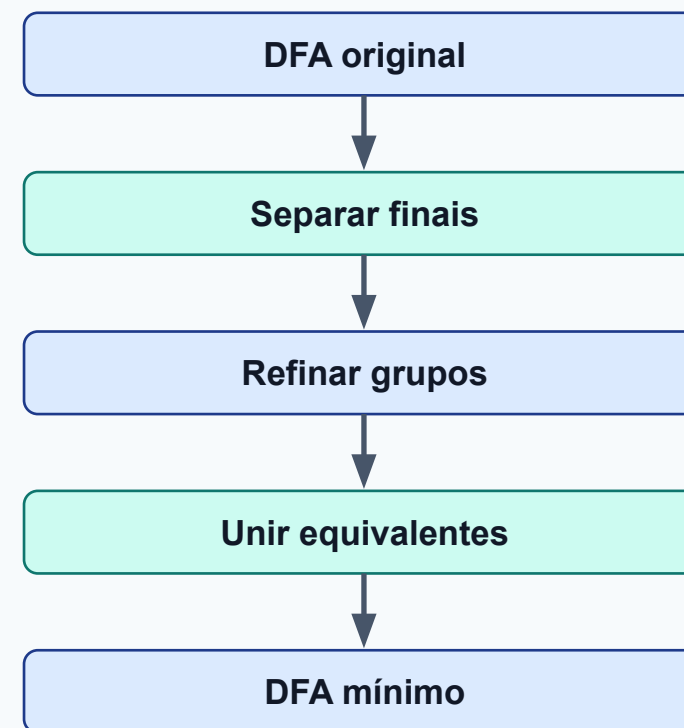
DFA: até 2^n estados

pior caso

Minimização de DFA

Redução sem mudar a linguagem

Depois de converter ou construir um DFA, pode haver estados redundantes. Minimizar o DFA significa agrupar estados que se comportam da mesma maneira diante de todas as entradas futuras. O resultado reconhece a mesma linguagem com menos estados, tornando o modelo mais simples de entender e implementar.



JFLAP como ferramenta de apoio

Aplicação gratuita para uso acadêmico

O JFLAP é uma ferramenta gratuita muito usada em disciplinas de linguagens formais e autômatos. Ela permite criar autômatos finitos, simular entradas, destacar não determinismo, destacar transições λ e realizar conversões. O uso visual ajuda a validar respostas que, no papel, seriam mais trabalhosas de conferir.

[JFLAP oficial — https://www.jflap.org/](https://www.jflap.org/)

Usando JFLAP

- 1 Acessar [jflap.org](https://www.jflap.org/)
- 2 Baixar o arquivo .jar
- 3 Abrir com Java
- 4 Escolher Finite Automaton
- 5 Salvar o arquivo .jff

Passo a passo: criar um DFA

Prática no JFLAP

Para criar um DFA no JFLAP, abra a ferramenta, escolha Finite Automaton, crie os estados, marque o estado inicial, marque os finais e desenhe as transições. Em seguida, use a simulação para testar palavras aceitas e rejeitadas. O objetivo é conferir se o desenho reconhece a linguagem pretendida.

[Tutorial JFLAP FA — https://www.jflap.org/tutorial/fa/createfa/fa.html](https://www.jflap.org/tutorial/fa/createfa/fa.html)

DFA no JFLAP

- 1 File > New
- 2 Finite Automaton
- 3 Criar estados
- 4 Definir inicial/final
- 5 Inserir transições

Passo a passo: testar entradas

Multiple Run

Depois de montar o autômato, use Input > Multiple Run para testar várias entradas ao mesmo tempo. Digite uma palavra por linha e execute. A ferramenta mostrará Accept ou Reject. Esse procedimento é excelente para encontrar transições esquecidas, estados finais marcados incorretamente e interpretações erradas da linguagem.

Testes

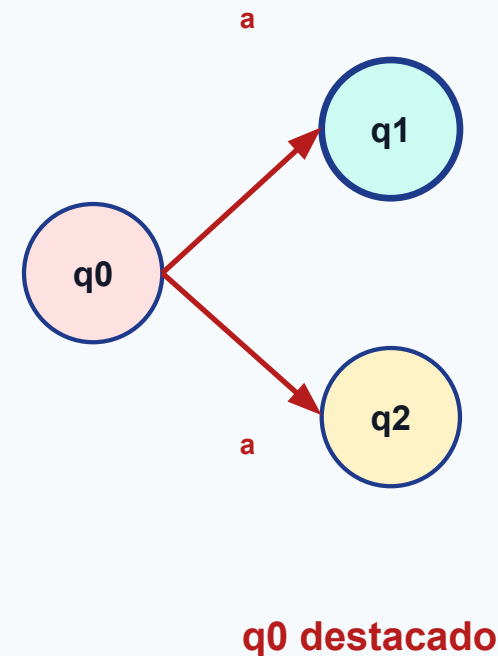
Entrada	Esperado	JFLAP
b	aceita	Accept
ab	aceita	Accept
a	rejeita	Reject
ba	rejeita	Reject

Passo a passo: localizar não determinismo

Highlight Nondeterminism

No JFLAP, é possível pedir que a ferramenta destaque estados não determinísticos. Isso ocorre quando há mais de uma transição possível para o mesmo símbolo, ou quando certas escolhas tornam a execução múltipla. Esse recurso ajuda a diferenciar visualmente um DFA de um NFA durante a correção de exercícios.

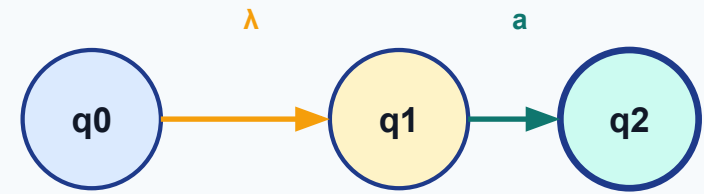
[Tutorial JFLAP FA — https://www.jflap.org/tutorial/fa/createfa/fa.html](https://www.jflap.org/tutorial/fa/createfa/fa.html)



Passo a passo: transições λ

Highlight λ -Transitions

Em autômatos com transições vazias, use a opção de destaque de λ -transitions para localizar mudanças de estado que não consomem símbolos. Durante a simulação, observe que a entrada restante não muda quando a transição λ ou ϵ é tomada. Esse cuidado evita erros ao rastrear caminhos.



entrada não é consumida

Prática com Python

Simulador simples de DFA

Além do JFLAP, é possível implementar um simulador de DFA em Python usando dicionários. Cada chave representa um par estado-símbolo e cada valor indica o próximo estado. Essa prática ajuda o aluno a perceber que o diagrama pode ser traduzido diretamente em estrutura de dados.

[Python docs — https://docs.python.org/3/](https://docs.python.org/3/)

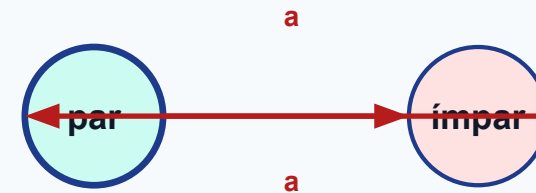
DFA em Python

```
delta = {('q0','a'):'q0',  
        ('q0','b'):'q1',  
        ('q1','a'):'q0',  
        ('q1','b'):'q1'}  
estado = 'q0'  
for c in 'aab':  
    estado = delta[(estado,c)]  
print(estado in {'q1'})
```

Exercício guiado 1

Construir DFA para palavras com número par de a

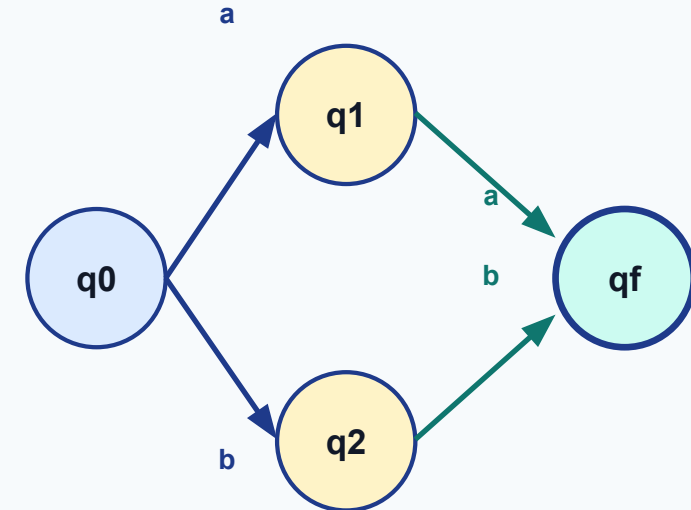
Construa um DFA sobre $\{a,b\}$ que aceite palavras com quantidade par de ocorrências de a. Use dois estados: q_0 para quantidade par e q_1 para quantidade ímpar. O símbolo b não muda a paridade. Marque q_0 como inicial e final, pois zero ocorrências de a também é quantidade par.



Exercício guiado 2

Construir NFA com alternativa

Construa um NFA que aceite palavras que começam com ab ou com ba. O estado inicial pode ter dois caminhos: um para tentar reconhecer ab e outro para tentar reconhecer ba. Depois de reconhecer uma das alternativas, o autômato pode ir para estado final. Teste ab, ba, aa, bb e aba.



Erros comuns

Atenção na resolução de exercícios

Os erros mais frequentes são esquecer o estado inicial, marcar estado final errado, confundir ϵ com símbolo da entrada, usar duas transições iguais em um DFA, deixar transições importantes ausentes e aceitar a palavra antes de consumir toda a entrada. Testes sistemáticos costumam revelar rapidamente essas falhas.

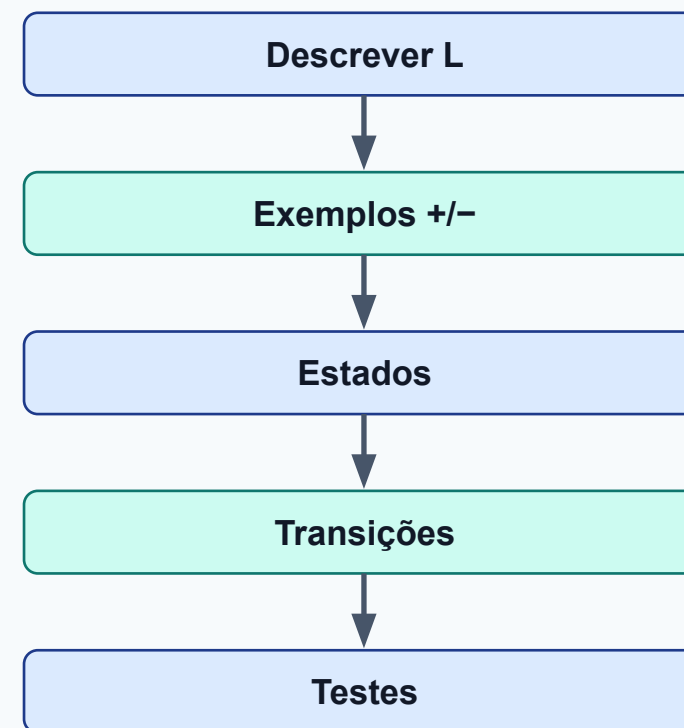
Checklist

Erro	Como detectar	Correção
sem final	nada aceita	marcar F
ϵ errado	entrada muda	não consumir
DFA ambíguo	duas setas	separar NFA
faltou teste	aceita errado	testar casos

Roteiro de estudo

Como praticar melhor

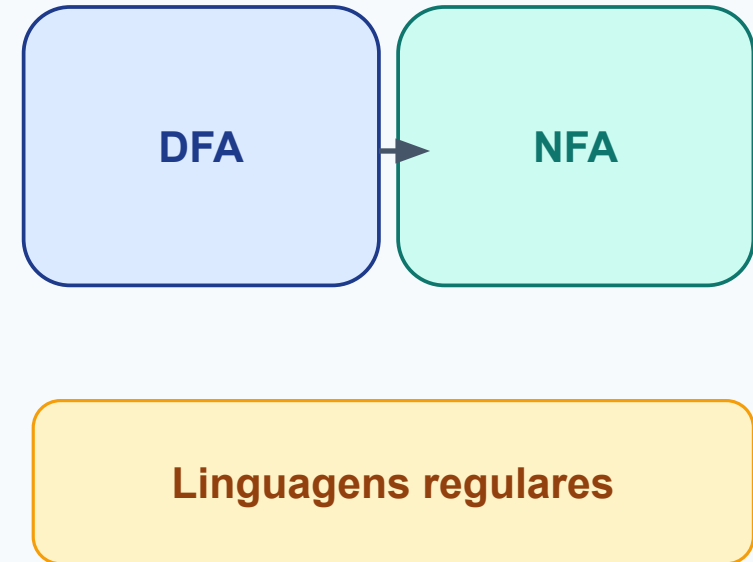
Para estudar autômatos, descreva primeiro a linguagem em português, liste exemplos aceitos e rejeitados, defina quais informações precisam ser lembradas, desenhe os estados, complete as transições e teste entradas variadas. Depois, tente representar o mesmo problema por tabela, diagrama e, quando possível, código.



Síntese da aula

Ideias essenciais

DFA segue um único caminho e é direto para implementar. NFA permite múltiplos caminhos e transições vazias, facilitando a modelagem. Transições ϵ mudam de estado sem consumir entrada. Apesar das diferenças, DFA e NFA reconhecem as mesmas linguagens regulares e são fundamentais para compreender analisadores léxicos.



Referências e materiais recomendados

Fontes utilizadas

Plano de Ensino da disciplina Linguagens Formais e Compiladores 2026/2. Rodger e Finley, JFLAP: An Interactive Formal Languages and Automata Package, 2005. JFLAP oficial e tutorial de autômatos finitos. Documentação Python sobre expressões regulares. MDN Web Docs sobre regular expressions. OWASP ReDoS para cuidados

contemporâneos com regex.

JFLAP
Python re
MDN Regex
OWASP ReDoS
Plano de Ensino